🔧 **Purpose:**

To read raw data, split it into train and test sets, save the data, and then run data transformation and model training components.

🧱 **Components & Flow:**

1. DataIngestionConfig

- A configuration class that defines file paths to store:
    - Raw data
    - Training data
    - Test data

2. DataIngestion Class

- Handles reading and splitting data.
- Key method:
- initiate_data_ingestion()
    - Reads stud.csv file from notebook/data/
    - Saves the raw data
    - Splits it into train and test sets
    - Saves them to artifacts/train.csv and artifacts/test.csv
    - Returns paths to train and test data files

3. Main Block:

if __name__=="__main__":

- Runs the full pipeline:
    1. **Ingest data**
    2. obj=DataIngestion()
    3. train_data, test_data = obj.initiate_data_ingestion()
    4. **Transform data**
    5. data_transformation = DataTransformation()
    6. train_arr, test_arr, _ = data_transformation.initiate_data_transformation(train_data, test_data)
    7. **Train model**
    8. modeltrainer = ModelTrainer()
    9. print(modeltrainer.initiate_model_trainer(train_arr, test_arr))

🧱 **Dependencies Used:**

- pandas: For reading and manipulating data
- sklearn: For splitting the dataset
- logging: For logging pipeline steps
- dataclasses: For clean configuration structure
- CustomException: For consistent error handling
- Modules like DataTransformation, ModelTrainer: Handle transformation and model training (presumably from other files in src/components/)

✅ **Summary:**

The script:

1. Reads raw student data (stud.csv)
2. Splits and saves train/test data
3. Transforms the data
4. Trains a model and prints the result

Line by line explanation:

```
import os
import sys
from src.exception import CustomException
from src.logger import logging
import pandas as pd
```

os and sys: Used for file system operations and exception handling.
CustomException: A custom error-handling class (likely defined in src/exception.py).
logging: A custom logger (likely defined in src/logger.py) used to log information during pipeline execution.
pandas as pd: For reading and manipulating tabular data (.csv files in this case).

```
from sklearn.model_selection import train_test_split
from dataclasses import dataclass
```

train_test_split: A Scikit-learn function used to split the dataset into training and testing sets.

dataclass: A decorator to simplify class definitions used mainly for storing data (like configs).

```
from src.components.data_transformation import DataTransformation
from src.components.data_transformation import DataTransformationConfig

from src.components.model_trainer import ModelTrainerConfig
from src.components.model_trainer import ModelTrainer
```

The data transformation logic and its config.

The model trainer and its config.

These are part of the pipeline and assumed to be defined elsewhere in the src/components/ folder.

DataIngestionConfig class

```
@dataclass
class DataIngestionConfig:
    train_data_path: str = os.path.join('artifacts', "train.csv")
    test_data_path: str = os.path.join('artifacts', "test.csv")
    raw_data_path: str = os.path.join('artifacts', "data.csv")
```

@dataclass: Automatically generates __init__, __repr__, etc., for this class.

This class holds the file paths where:
Raw data is saved (data.csv)
Train and test data are saved after splitting (train.csv and test.csv)

All files go into the artifacts/ directory.

DataIngestion class

```
class DataIngestion:
    def __init__(self):
        self.ingestion_config = DataIngestionConfig()
```

Initializes an object of DataIngestionConfig and stores it as self.ingestion_config.

initiate_data_ingestion Method

```
def initiate_data_ingestion(self):

    logging.info("Entered the data ingestion method or component")
```

Logs that the data ingestion process has started.

```
    try:
        df = pd.read_csv('notebook\data\stud.csv')
        logging.info('Read the dataset as dataframe')
```

Tries to read a CSV file (stud.csv) from notebook/data/ and stores it in df.

Logs that reading was successful.

os.makedirs(os.path.dirname(self.ingestion_config.train_data_path), exist_ok=True)

Creates the directory artifacts/ if it doesn't already exist, to store output files.

df.to_csv(self.ingestion_config.raw_data_path, index=False, header=True)

Saves the entire dataset as a raw CSV (data.csv).

logging.info("Train test split initiated")
train_set, test_set = train_test_split(df, test_size=0.2, random_state=42)

Logs that splitting is starting.

Splits the dataset: 80% training data, 20% test data.

train_set.to_csv(self.ingestion_config.train_data_path, index=False, header=True)
test_set.to_csv(self.ingestion_config.test_data_path, index=False, header=True)

Saves the train_set and test_set as CSV files in artifacts/.

logging.info("Ingestion of the data iss completed")

Logs that the data ingestion process has finished.

return (
self.ingestion_config.train_data_path,
self.ingestion_config.test_data_path
)

Returns paths to the train and test CSV files.

except Exception as e:
raise CustomException(e, sys)

If anything goes wrong, it raises a custom exception for better debugging and error tracking.

🚀 **Main Execution Block**

if __name__ == "__main__":

This block runs only when the script is executed directly (not when imported).

obj = DataIngestion()
train_data, test_data = obj.initiate_data_ingestion()

Creates an object of DataIngestion class and runs the ingestion process.
Gets paths to the train and test datasets.

data_transformation = DataTransformation()

train_arr, test_arr, _ = data_transformation.initiate_data_transformation(train_data, test_data)

Calls the data transformation component.

Likely performs feature scaling, encoding, etc., and returns numpy arrays for training/testing.

modeltrainer = ModelTrainer()

print(modeltrainer.initiate_model_trainer(train_arr, test_arr))

Calls the model training component.

Trains a machine learning model using transformed data.

Prints the result (could be accuracy, R^2 score, etc.).

✅ Summary:

Step	What Happens
1.	Dataset is loaded from CSV
2.	Raw data is saved
3.	Data is split into train/test
4.	Train/test CSVs are saved
5.	Data is transformed
6.	Model is trained and result is printed